

If a Broken Clock is Right Twice a Day

Arian Nazarian



Figure 1: Damaged Omega Speedmaster 3221.30.00 — its civil-time and calendar indications frozen at 10:08:00 on Saturday, March 11. The small-seconds subdial also carries a 24-hour hand, which distinguishes the two halves of the day⁴.

Abstract

What is the probability of randomly checking a broken time-only 12-hour watch and discovering that it matches the civil time at the sampled instant? If the dial has no AM/PM indicator, the calculation is a simple finite-counting problem: at any fixed display resolution, the same 12-hour reading corresponds to two states in a 24-hour cycle. What if the watch is an Omega Speedmaster 3221.30.00 "Day-Date," whose civil-time display includes a 24-hour indication in addition to month, weekday, and date? How often does such a configuration align with reality, and what is the probability of a strict civil-time-and-calendar match when checked randomly?

These seemingly trivial questions may suggest inconsequential solutions to the uninitiated, but consider a traveler that is moored in time, bound not by will but by a watch standing still. The hands are stuck, the calendar unchanged, yet this broken mechanism serves as their only guide. They may only journey to the rare intersections where time's machinery meets its motionless twin.

The irregular yet cyclical structure of the Gregorian calendar presents a finite but non-uniform recurrence problem. This paper explores the calendar's underlying mathematical framework, compares algorithmic approaches to computing date recurrences, and gives exact probabilities for when

a fixed date-time display recurs. The main calculation is carried out over the Gregorian 400-year cycle, whose 146,097 days form the natural sample space for the problem.

1 The Gregorian Calendar

The Gregorian calendar was promulgated in 1582 as a reform of the Julian calendar; its adoption thereafter varied by jurisdiction.^{1;6} Its modern date representations are standardized in ISO 8601.³

The Julian system has an average year length of

$$365.25 \text{ days,}$$

whereas the mean tropical year is approximately

$$365.2422 \text{ days.}$$

The difference is therefore

$$365.25 - 365.2422 \approx 0.0078 \text{ days/year.}$$

Over the centuries this discrepancy displaced the ecclesiastical equinox convention from 21 March. In jurisdictions implementing the Gregorian reform in 1582, ten nominal calendar dates were omitted in order to restore the intended alignment.⁶

The Gregorian calendar also introduced a modified leap-year rule:^{1;6}

- every year divisible by 4 is a leap year;
- except years divisible by 100, which are not leap years;
- unless they are also divisible by 400, in which case they remain leap years.

Thus a 400-year cycle contains 97 leap years and 303 common years. Its average year length is

$$\frac{(303 \times 365) + (97 \times 366)}{400} = 365.2425 \text{ days,}$$

which differs from the mean tropical year by about 0.0003 day per year, or roughly one day in 3300 years.⁶

More importantly for weekday recurrence, the same 400-year block contains

$$D = (303 \times 365) + (97 \times 366) = 146\,097 = 20\,871 \times 7$$

days. The leap-year pattern itself repeats every 400 years, and D is an integer number of weeks. Consequently, the assignment of weekdays to valid Gregorian month–date pairs repeats exactly after 400 years. This 400-year period is therefore the natural cycle for the recurrence calculations below.^{1;6}

2 Zeller's Congruence: Computing the Day of the Week

Zeller's congruence is an efficient algorithm for determining the weekday of any Gregorian date.⁷ The congruence is a weekday rule on valid civil dates; it does not by itself certify that an arbitrary numerical year–month–day triple represents an existing Gregorian date. It is expressed as

$$h = \left(q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor - 2J \right) \bmod 7, \quad (1)$$

where

- h is the weekday (0 = Saturday, 1 = Sunday, ..., 6 = Friday);
- q is the day of the month;
- m is the *modified* month: March = 3, ..., December = 12, January = 13, February = 14 (treated as months of the previous year);
- $K = Y \bmod 100$ (year within the century);
- $J = \lfloor Y/100 \rfloor$ (century).

Handling January and February

Because January and February are counted as months 13 and 14 of the *previous* year, the calendar year must be decremented by one whenever $m > 12$:

$$\text{if } m > 12, \quad Y \leftarrow Y - 1.$$

This shift automatically aligns leap-year logic and century boundaries.

Alternative Formulation: Starting Months at One

If months use the conventional numbering (January = 1, ..., December = 12), let M be that month index. Zeller's formula becomes⁷

$$h = \left(q + \left\lfloor \frac{13(M+1+12L)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor - 2J \right) \bmod 7,$$

where

$$L = \begin{cases} 1, & M \leq 2 \text{ (January or February)}, \\ 0, & \text{otherwise,} \end{cases} \quad K = (Y - L) \bmod 100, \quad J = \lfloor (Y - L)/100 \rfloor.$$

The adjustment term L explicitly handles the leap-year boundary, albeit at the cost of a slightly more complex expression.

3 Direct Derivation from Gregorian Rules

An alternative approach derives the weekday directly from the Gregorian calendar's definitions. For a valid proleptic Gregorian date with civil year $Y \geq 1$, the method counts the number of days elapsed since 1 January of year 1 and reduces that count modulo 7.^{1;6}

Constructing the Formula

Using q to denote the day of the month, the weekday can be computed by counting all elapsed days before the date and reducing modulo 7:

$$h = (h_0 + (q - 1) + T(Y) + M_{\text{offset}}(M, Y)) \bmod 7,$$

where:

- h represents the day of the week under the chosen numbering convention.
- h_0 is the weekday of 1 January in year 1 under that convention. With Zeller's convention $0 = \text{Saturday}$, $1 = \text{Sunday}$, ..., $6 = \text{Friday}$, one has $h_0 = 2$, since 1 January 1 is Monday in the proleptic Gregorian calendar.
- $T(Y)$ represents the total number of days from all full years before Y .
- $M_{\text{offset}}(M, Y)$ represents the total number of days from completed months in year Y .

Counting Days from Full Years

The total number of days contributed by past years before year Y is:

$$T(Y) = (Y - 1) \times 365 + C(Y - 1).$$

Here, for $n \geq 0$, $C(n)$ counts the leap years among years $1, 2, \dots, n$ (with $C(0) = 0$):

$$C(n) = \lfloor n/4 \rfloor - \lfloor n/100 \rfloor + \lfloor n/400 \rfloor.$$

For the current year itself, it is useful to separate the cumulative count $C(n)$ from the leap-year indicator

$$\ell(Y) = \begin{cases} 1, & 4 \mid Y \text{ and } (100 \nmid Y \text{ or } 400 \mid Y), \\ 0, & \text{otherwise.} \end{cases}$$

Accumulating Days from Completed Months

The cumulative number of days contributed by past months in year Y is given by:

$$M_{\text{offset}}(M, Y) = \begin{cases} 0, & M = 1 \text{ (January)} \\ 31, & M = 2 \text{ (February)} \\ 31 + 28 + \ell(Y), & M = 3 \text{ (March)} \\ 31 + 28 + \ell(Y) + 31, & M = 4 \text{ (April)} \\ 31 + 28 + \ell(Y) + 31 + 30, & M = 5 \text{ (May)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31, & M = 6 \text{ (June)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30, & M = 7 \text{ (July)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31, & M = 8 \text{ (August)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31 + 31, & M = 9 \text{ (September)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31 + 31 + 30, & M = 10 \text{ (October)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31 + 31 + 30 + 31, & M = 11 \text{ (November)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31 + 31 + 30 + 31 + 30, & M = 12 \text{ (December)} \end{cases}$$

For months beyond February, the function accounts for leap years by adding the indicator $\ell(Y)$, not the cumulative leap-year count $C(Y)$. Although the piecewise definition of $M_{\text{offset}}(M, Y)$ appears unwieldy, it clearly enumerates the cumulative days leading up to each month.

Why Modulo 7 Determines the Weekday

Since the formula ultimately relies on taking the result modulo 7, let us examine why this step works:

- The sequence of weekdays follows a repeating cycle every 7 days.
- A normal year contains 365 days, which shifts the weekday forward by $365 \bmod 7 = 1$.
- A leap year contains 366 days, shifting the weekday forward by $366 \bmod 7 = 2$.
- The leap-year indicator $\ell(Y)$ adjusts for the extra day in leap years, preventing long-term drift.

Thus, by summing the total days and computing modulo 7, we effectively “wrap around” to obtain the correct weekday.

Comparison with Zeller’s Congruence

While both this direct derivation and Zeller’s congruence ultimately determine the day of the week, they differ in approach and computational efficiency.

- The direct derivation explicitly follows the Gregorian calendar’s structure, computing total elapsed days from a fixed reference date. It provides a conceptual and systematic understanding of how weekdays cycle over time but requires separate handling of year, month, and day contributions.
- Zeller’s congruence, on the other hand, is a more compact formula that applies modular arithmetic directly to a given date without explicitly summing total elapsed days. It incorporates a pre-adjusted month numbering system and operates in a form that is computationally efficient for quick lookups.
- The direct derivation is particularly useful for historical and educational purposes, as it breaks down the logic step by step, whereas Zeller’s congruence is optimized for direct calculation in programming and algorithmic implementations.

This derivation follows the structure of the Gregorian calendar directly, ensuring an intuitive calculation of the weekday for any given date. It highlights how ordinary years advance the weekday by one, leap years advance it by two, and the Gregorian correction terms preserve a repeating structure modulo 7—a fundamental aspect also utilized elegantly in Zeller’s congruence.^{1:6}

4 Using Zeller's Congruence to Find Matching Years

Zeller's congruence is usually evaluated forward: a valid civil date is supplied and its weekday is returned. For a fixed month, day, and target weekday, the same congruence can instead be used as an exact predicate on candidate years.⁷

Calendar Year and Adjusted Year

Let $Y \geq 1$ denote the civil year, $M \in \{1, \dots, 12\}$ the conventional month number, and q the day of the month. Define

$$L = \begin{cases} 1, & M \leq 2, \\ 0, & M \geq 3, \end{cases} \quad Y^* = Y - L, \quad m = M + 12L.$$

Thus $m = 13$ and 14 for January and February, respectively, while $m = M$ from March through December. The adjusted quantity Y^* is the year used internally by Zeller's arithmetic. Set

$$K = Y^* \bmod 100, \quad J = \left\lfloor \frac{Y^*}{100} \right\rfloor.$$

For a target weekday h_{target} in Zeller's numbering, a valid candidate date (Y, M, q) matches precisely when

$$h_{\text{target}} \equiv q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor - 2J \pmod{7}.$$

Isolating the Year-Dependent Terms

Substituting $K = Y^* \bmod 100$ and $J = \lfloor Y^*/100 \rfloor$ gives

$$\begin{aligned} (Y^* \bmod 100) + \left\lfloor \frac{Y^* \bmod 100}{4} \right\rfloor + \left\lfloor \frac{\lfloor Y^*/100 \rfloor}{4} \right\rfloor - 2 \left\lfloor \frac{Y^*}{100} \right\rfloor \\ \equiv h_{\text{target}} - q - \left\lfloor \frac{13(m+1)}{5} \right\rfloor \pmod{7}. \end{aligned}$$

For fixed M , q , and h_{target} , the right-hand side is constant; the left-hand side depends only on the appropriately adjusted candidate year.

Civil-date validity is logically prior to weekday evaluation. In particular, a query for 29 February must skip non-leap years, and an impossible month-day pair such as 31 April must never be treated as a date merely because a modular expression returns a residue.

Algorithm for Finding Matching Years

For an inclusive range of positive civil years, an exact scan is therefore:

1. verify that the target weekday and month-day specification are admissible;
2. iterate through the candidate years;
3. skip any year in which the specified civil date does not exist;
4. form Y^* , m , K , and J , evaluate Zeller's congruence, and record the year if the target weekday is obtained.

The running time is linear in the number of years scanned and constant in memory apart from the returned matches. For recurrence frequencies, a complete 400-year cycle is sufficient because the Gregorian calendar state repeats after that period.

5 Comparison of Algorithm Implementations

This section contrasts two exact approaches for locating every year in which a given weekday, month, and date coincide in the Gregorian calendar: a Python routine that evaluates Zeller’s congruence directly⁷ and a JavaScript routine that uses the standardized ECMAScript `Date` object in UTC.²

Feature	Python (Zeller)	JavaScript (UTC Date)
Month numbering	March = 3, ..., December = 12; Jan = 13, Feb = 14 (previous year)	Jan = 0, ..., December = 11 internally
Weekday numbering	0 = Saturday, ..., 6 = Friday	0 = Sunday, ..., 6 = Saturday
Leap-year arithmetic	Explicit Gregorian rule plus Zeller’s century terms	ECMAScript Gregorian date arithmetic
Date validity	Explicit civil-date test before Zeller evaluation	Exact UTC year/month/day checked after construction
Century handling	Jan/Feb year adjustment and $J = \lfloor Y^*/100 \rfloor$	Built into <code>Date</code> arithmetic
Computation style	Integer modular arithmetic	Standardized UTC date operations
Use case	Direct scans and independent arithmetic verification	Browser-based or general JavaScript date lookup

Table 1: Python implementation of Zeller’s congruence vs. ECMAScript `Date`.

Both methods return the same matching years when supplied equivalent valid Gregorian inputs, after accounting for their different weekday numberings.

Python Script with Zeller's Congruence

The Python implementation keeps civil-date validity separate from weekday arithmetic. This distinction is necessary for a general month–day query: 29 February is valid only in Gregorian leap years, whereas a pair such as 31 April is invalid in every year.

Listing 1: Validated Python implementation of Zeller's congruence for matching years

```
def is_gregorian_leap_year(year):
    """Return whether a positive civil year is a Gregorian leap year."""
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)

def is_possible_month_day(month, day):
    """Return whether (month, day) exists in at least one Gregorian year."""
    if not (1 <= month <= 12 and day >= 1):
        return False
    maximum = (31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
    return day <= maximum[month - 1]

def is_valid_gregorian_date(year, month, day):
    """Return whether (year, month, day) is a valid positive Gregorian date."""
    if year < 1 or not is_possible_month_day(month, day):
        return False
    if month == 2 and day == 29:
        return is_gregorian_leap_year(year)
    return True

def zellers_congruence(day, month, year):
    """Return 0=Saturday, 1=Sunday, ..., 6=Friday for a valid date."""
    if not is_valid_gregorian_date(year, month, day):
        raise ValueError("Invalid Gregorian date.")

    if month < 3:
        month += 12
        year -= 1

    K = year % 100
    J = year // 100
    return (day + (13 * (month + 1)) // 5 + K + K // 4 + J // 4 - 2 * J) % 7

def find_matching_years(target_weekday, month, day, start_year, end_year):
    """Return years in an inclusive range matching a weekday-month-day triple."""
    values = (target_weekday, month, day, start_year, end_year)
    if not all(isinstance(x, int) and not isinstance(x, bool) for x in values):
        raise TypeError("All arguments must be integers.")
    if not 0 <= target_weekday <= 6:
        raise ValueError("target_weekday must be in 0,...,6.")
    if start_year < 1 or end_year < start_year:
        raise ValueError("Require 1 <= start_year <= end_year.")
    if not is_possible_month_day(month, day):
        raise ValueError("The month-day pair is impossible in the Gregorian calendar.")

    matches = []
    for year in range(start_year, end_year + 1):
        if is_valid_gregorian_date(year, month, day):
            if zellers_congruence(day, month, year) == target_weekday:
                matches.append(year)
    return matches

if __name__ == "__main__":
    # March 11 on Saturday (Zeller numbering: Saturday = 0)
    print(find_matching_years(0, 3, 11, 2001, 2100))
    # [2006, 2017, 2023, 2028, 2034, 2045, 2051, 2056, 2062, 2073, 2079, 2084, 2090]
```

The implementation uses only the Gregorian divisibility rule and Zeller's congruence; it does not call a date-time library to obtain the weekday. The example searches 2001–2100 for years in which 11 March is a Saturday.

JavaScript Using the Date Object

The ECMAScript `Date` object performs Gregorian date arithmetic, and UTC accessors make the result independent of the host machine's local time zone.² Two details matter for a general implementation. First, out-of-range date components are normalized by the date algorithms, so the constructed UTC fields must be checked explicitly rather than assuming that construction certifies civil-date validity. Second, `Date.UTC` applies the specification's special full-year interpretation to numerical years $0, \dots, 99$. The implementation therefore uses `setUTCFullYear`, which accepts the supplied year directly, and then verifies the resulting UTC fields.

Listing 2: Validated JavaScript implementation using the built-in `Date` object

```
const MAXIMUM_DAY_BY_MONTH = Object.freeze(
  [31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
);

function isPossibleMonthDay(month, day) {
  if (month < 1 || month > 12 || day < 1) return false;
  return day <= MAXIMUM_DAY_BY_MONTH[month - 1];
}

function utcDateOrNull(year, month, day) {
  // setUTCFullYear interprets years 0,...,99 literally, unlike Date.UTC.
  const date = new Date(0);
  date.setUTCFullYear(year, month - 1, day);

  if (!Number.isFinite(date.getTime())) {
    throw new RangeError("Requested date is outside the ECMAScript Date range.");
  }

  const exactDate =
    date.getUTCFullYear() === year &&
    date.getUTCMonth() === month - 1 &&
    date.getUTCDate() === day;

  return exactDate ? date : null;
}

function findMatchingYears(targetWeekday, month, day, startYear, endYear) {
  const values = [targetWeekday, month, day, startYear, endYear];
  if (!values.every(Number.isSafeInteger)) {
    throw new TypeError("All arguments must be safe integers.");
  }
  if (targetWeekday < 0 || targetWeekday > 6) {
    throw new RangeError("targetWeekday must be in 0,...,6.");
  }
  if (startYear < 1 || endYear < startYear) {
    throw new RangeError("Require 1 <= startYear <= endYear.");
  }
  if (!isPossibleMonthDay(month, day)) {
    throw new RangeError("The month-day pair is impossible in the Gregorian calendar.");
  }

  const matches = [];
  for (let year = startYear; year <= endYear; year++) {
    const date = utcDateOrNull(year, month, day);
    if (date !== null && date.getUTCDay() === targetWeekday) {
      matches.push(year);
    }
  }
  return matches;
}

// March 11 on Saturday (ECMAScript numbering: Saturday = 6)
console.log(findMatchingYears(6, 3, 11, 2001, 2100));
// [2006, 2017, 2023, 2028, 2034, 2045, 2051, 2056, 2062, 2073, 2079, 2084, 2090]
```

The JavaScript function rejects inputs outside its stated interface, skips candidate years in which a possible month-day pair does not exist, and raises an error if a requested civil date is outside the representable ECMAScript `Date` range.

Exhaustive Cross-Validation

The two implementations were cross-checked over every valid civil date in years 1 through 400, one complete Gregorian cycle. After translating the weekday-numbering conventions, the Python Zeller routine, the JavaScript UTC routine, and the direct day-count construction of Section 3 agreed on all

$$146\,097$$

valid dates. Additional boundary checks distinguish leap-century cases correctly: 29 February exists in 2000 but not in 1900, and a normalized JavaScript construction is accepted only when its resulting UTC year, month, and date exactly match the requested civil date. This exhaustive cycle check does not replace the mathematical derivations; it verifies that the supplied implementations realize them consistently.

6 Probabilistic Analysis, Applications, and Conclusion

Foundations: Probability Tools for Fixed Date–Time Recurrences

The probability calculation is a finite counting problem. Let $\mathcal{D}_{400}$ be the set of valid civil dates in one complete Gregorian 400-year cycle, so

$$|\mathcal{D}_{400}| = D = 146\,097.$$

At second-level precision let

$$\mathcal{S} = \{0, 1, \dots, 86\,399\}$$

index the seconds of a modeled civil day. The finite state space is

$$\Omega = \mathcal{D}_{400} \times \mathcal{S}, \quad |\Omega| = N = D \times 86\,400 = 12\,622\,780\,800.$$

Scope of the model. The Gregorian calendar is treated as a deterministic periodic system in which each modeled civil day contains exactly 86 400 second-level states. Time zones, daylight-saving adjustments, leap seconds, and jurisdiction-specific adoption dates are excluded; including them would define a different timekeeping state space rather than change the Gregorian recurrence counts themselves. The uniform distribution on Ω is a stipulated sampling model for a “random glance,” not a claim about the empirical distribution of human observation times.

If $A \subseteq \Omega$ is the event that the frozen display matches the sampled state, then

$$\Pr(A) = \frac{|A|}{|\Omega|}.$$

If repeated glances are modeled as independent draws from this same uniform distribution, let $p = \Pr(A)$. If G is the number of glances through the first match, then G is geometric:

$$\Pr(G = g) = (1 - p)^{g-1}p, \quad E[G] = \frac{1}{p}, \quad \text{Var}(G) = \frac{1 - p}{p^2}, \quad g = 1, 2, \dots$$

This geometric interpretation concerns independent random inspections. It is not the same as observing the watch continuously or at fixed deterministic intervals, where successive observations are not independent.⁵

Time-Only Watches (12-Hour Dial)

Let the frozen dial read $h:m:s$. A 12-hour dial without an AM/PM indicator has two corresponding occurrences in each 24-hour cycle. At a coarser display resolution, each occurrence represents the interval during which that displayed value remains unchanged; equivalently, one may count uniformly over the corresponding hour-, minute-, or second-level states. Thus the probabilities below are unchanged whether they are obtained from the appropriate coarse state space or by counting the equivalent number of second-level states.

Display precision	Matching states per 24-hour cycle	p
Hours	2	$2/24 = 1/12$
Hours + Minutes	2	$2/(24 \times 60) = 1/720$
Hours + Min + Sec	2	$2/(24 \times 60 \times 60) = 1/43\,200$

The factor “2” is the familiar AM/PM duplication. A 24-hour dial, or a display that explicitly distinguishes AM from PM, halves each probability.

Adding the Calendar Fields

The sample space remains the set of N second-level states in the 400-year Gregorian cycle. Adding calendar fields changes the success event, not the underlying sample space. The month and day specify a month–date pair; the weekday is then determined by the actual calendar date. Thus a display such as

Saturday, 11 March, 10:08:00

matches only in years for which 11 March is a Saturday. If the time display is an AM/PM-ambiguous 12-hour dial, there are two matching 10:08:00 states on each qualifying date. If the display also distinguishes the half-day, only one of those two states can match.

Probability of the Sample Display

The Omega Speedmaster in Fig. 1 has central hour and minute hands, a small-seconds indication, and calendar indications for date, weekday, and month. Its small-seconds subdial also carries a 24-hour hand. Consequently, although the central hour indication is 12-hour, the civil-time display distinguishes the two halves of the day.⁴

For the probability calculation, the *strict civil-time-and-calendar match* compares the frozen hour, minute, small-seconds indication, 24-hour indication, weekday, month, and date with the corresponding civil-time state. Chronograph elapsed-time indications are excluded: their readings depend on an externally initiated timing operation rather than on civil time. Under this definition the frozen civil-time-and-calendar state is

Saturday, 11 March, 10:08:00 with the displayed 24-hour state.

400-year theoretical cycle.

- **Weekday–date matches.** A complete 400-year scan gives 11 March as a Saturday in 56 of the 400 years; the independent methods in Section 5 agree on this count.⁷
- **Strict civil-time-and-calendar match.** The 24-hour hand resolves the AM/PM ambiguity of the central 12-hour indication. Exactly one of the two daily central readings of 10:08:00 has the same 24-hour state as the frozen watch. Hence

$$s_{\text{strict}} = 56.$$

- **Calendar-only match (time ignored).** Ignoring the time of day gives

$$s_{\text{cal}} = 56 \times 86\,400 = 4\,838\,400$$

second-level states.

Therefore

$$p_{\text{strict}} = \frac{56}{12\,622\,780\,800} = \frac{1}{225\,406\,800} \approx 4.44 \times 10^{-9},$$

while

$$p_{\text{calendar}} = \frac{4\,838\,400}{12\,622\,780\,800} = \frac{56}{146\,097} \approx 3.83 \times 10^{-4} \approx \frac{1}{2609}.$$

Under the independent-random-glance model, the expected number of glances is $E[G_{\text{strict}}] = 225\,406\,800$ for the strict civil-time-and-calendar match and $E[G_{\text{calendar}}] \approx 2609$ for the calendar-only match.

Illustrative finite interval 1583–2024. As a finite-window comparison, treat the Gregorian rules as governing every date from 1 January 1583 through 31 December 2024. This interval contains 442 complete years and 161 438 days, so

$$N_{\text{win}} = 86\,400 \times 161\,438 = 13\,948\,243\,200.$$

In this interval,

$$\#(\text{Saturday, 11 March}) = 62,$$

so

$$s_{\text{strict, win}} = 62, \quad s_{\text{cal, win}} = 62 \times 86\,400.$$

Hence

$$p_{\text{strict, win}} = \frac{62}{13\,948\,243\,200} \approx 4.45 \times 10^{-9},$$

and

$$p_{\text{calendar, win}} = \frac{62}{161\,438} \approx 3.84 \times 10^{-4}.$$

The finite-window values lie close to the complete-cycle frequencies, as expected from the much longer periodic benchmark.

Most and Least Common (Weekday, Month, Date) Triples

Let a valid calendar triple mean a specified weekday, month, and date, with no time of day attached. The probability below is therefore a per-day probability over the $D = 146\,097$ days of the 400-year cycle.

Ideal 400-year cycle. For ordinary month–date pairs other than 29 February, each date occurs 400 times. The complete-cycle scan verified in Section 5 shows that, because of the placement of leap years within the 400-year period, a fixed ordinary month–date occurs on any specified weekday either 56, 57, or 58 times.

For 29 February, the date occurs only in the 97 leap years of the cycle; its weekday counts are 13, 14, or 15.

Thus the largest and smallest positive triple probabilities are

$$\boxed{p_{\text{max}} = \frac{58}{146\,097}}, \quad \boxed{p_{\text{min}} = \frac{13}{146\,097}}.$$

If a fixed second on an AM/PM-ambiguous 12-hour dial is also included, these become

$$\frac{2 \times 58}{12\,622\,780\,800} \quad \text{and} \quad \frac{2 \times 13}{12\,622\,780\,800},$$

respectively. If the time display distinguishes the half-day, the corresponding numerators are instead 58 and 13. Triples with impossible dates, such as 31 November, occur zero times and are excluded from the positive minimum.

Illustrative finite interval 1583–2024. With 442 complete years and 108 leap days in the same Gregorian interval,

$$\#_{\text{max, win}} = 65, \quad \#_{\text{min, win}} = 14.$$

Therefore

$$p_{\text{max, win}} = \frac{65}{161\,438}, \quad p_{\text{min, win}} = \frac{14}{161\,438}.$$

The finite window slightly changes the counts, but not the qualitative ranking: the largest counts occur among ordinary dates, while 29 February produces the smallest positive counts.

Applications and Implications

The preceding calculations have modest but concrete applications. First, the 400-year recurrence counts give exact benchmark values for calendar software. Since the Gregorian cycle contains 146 097 days and repeats modulo 7, a correct implementation of Gregorian weekday arithmetic should reproduce the same frequencies for fixed month–date–weekday triples, including 56 Saturday–11 March occurrences in the cycle. Disagreement would indicate an error in leap-year handling, century-year treatment, month indexing, or weekday numbering.^{1;6}

Second, the comparison between Zeller’s congruence, the direct day-count construction, and the ECMAScript `Date` object illustrates algorithmic cross-validation. Zeller’s formula is explicit modular arithmetic, the direct construction follows the calendar definition, and the JavaScript implementation delegates date arithmetic to a standardized object. Their exhaustive agreement over one complete 400-year cycle checks the implementation against the full recurrence period rather than against a small collection of examples. UTC methods additionally remove host-local time-zone effects from the JavaScript calculation.^{7;2}

Finally, the watch problem is a compact example of finite-state probability modeling. The physical display creates an event A inside a deterministic periodic state space Ω ; the probability is obtained by counting favorable states rather than assuming independence among calendar fields. This is the same modeling discipline used in finite probability problems more generally: define the sample space, specify the sampling rule, identify the event, and compute its probability from the relevant count.⁵ The main implication is therefore methodological rather than technological: irregular deterministic structures can still yield exact probabilities when the state space is finite and the recurrence law is known.

Conclusion

The broken-watch question is ultimately a finite-state probability problem. The Gregorian calendar supplies the state space; Zeller’s congruence supplies an efficient weekday test; and uniform counting supplies the probability. The key point is that the calendar fields are not independent decorations on the watch display. Once month and date are fixed, the weekday can occur only in those years where the Gregorian cycle permits that alignment.

For the damaged Omega Speedmaster display, 11 March falls on a Saturday 56 times in the 400-year Gregorian cycle. The watch’s 24-hour hand distinguishes the two otherwise identical 10:08:00 readings of a 12-hour central indication, so exactly one second on each qualifying date matches the frozen civil-time state. Therefore the exact strict civil-time-and-calendar probability under a random second-level glance is

$$\frac{56}{12\,622\,780\,800} = \frac{1}{225\,406\,800},$$

about one chance in 2.25×10^8 . If the time of day is ignored and only the calendar display is checked, the probability is

$$\frac{56}{146\,097},$$

about one chance in 2609. These values answer the original question without requiring approximation beyond the final decimal presentation.

References

- [1] Nachum Dershowitz and Edward M. Reingold. *Calendrical Calculations*. Cambridge University Press, Cambridge, UK, 3rd edition, 2008. See p. 45 for discussion of the Gregorian calendar reform and leap-year rules.
- [2] Ecma International. ECMAScript 2026 language specification (17th edition), 2026. URL <https://tc39.es/ecma262/2026/>. Standard. See especially the `Date` algorithms in Section 21.4.
- [3] International Organization for Standardization. ISO 8601-1:2019 — date and time — representations for information interchange — part 1: Basic rules, 2019. Standard.
- [4] Omega SA. Omega speedmaster date & day date chronograph 40 mm. <https://www.omegawatches.com/en-us/watch-omega-speedmaster-date-day-date-chronograph-40-mm-day-date-32213000>, n.d. Retrieved 3 September 2026. Image modified for illustrative purposes. Original image available at <https://www.omegawatches.com/media/catalog/product/o/m/omega-speedmaster-date--day-date-chronograph-40-mm-day-date-32213000-55a8b0.png?w=1100>. Omega is a registered trademark of Omega SA.
- [5] Sheldon M. Ross. *Introduction to Probability Models*. Academic Press, 12 edition, 2019. ISBN 9780128143469.
- [6] United States Naval Observatory. Introduction to calendars. <https://aa.usno.navy.mil/faq/calendars>, n.d. Retrieved 3 September 2026.
- [7] Christian Zeller. Kalender-formeln. *Acta Mathematica*, 9:131–136, 1886. doi: 10.1007/BF02406733.